

GoTrip – Technische Dokumentation

Modul: Mensch-Computer-Interaktion (HCI) – SoSe 2026, Meilenstein 3
Hochschule: h_da – Hochschule Darmstadt
Team: Mohamad Haj Ahmad · Wajdy Eleyan · Amal Najah · Eya Mathlouthi

1. Einleitung

GoTrip ist eine mobile Web-App, die Gruppen bei der gemeinsamen Reiseplanung unterstützt. Dieses Dokument beschreibt die technische Umsetzung des dritten Meilensteins: wie die in MS1 und MS2 erarbeiteten Spezifikationen und Prototypen in funktionierenden Code überführt wurden, welche Architekturentscheidungen getroffen wurden und welche Probleme im Entwicklungsprozess aufgetreten und gelöst wurden.

2. Umsetzung der Features

2.1 Abgleich mit den Spezifikationen (MS1/MS2)

Alle in den vorherigen Meilensteinen definierten Kernfeatures wurden implementiert:

Feature (Spezifikation)	Umsetzung	Status
Gruppenreise erstellen	CreateTrip.tsx + TripContext	✓
Mitglieder einladen	Einladungslink mit Base-57-Code	✓
Verfügbarkeit eintragen	AvailabilityCalendar mit 3 Zuständen	✓
Reisepräferenzen	Budget-Picker + 9 Interessen-Chips	✓
KI-Empfehlung	Score-Engine mit echten API-Daten	✓
Reiseziele bewerten	StarRating (0,5–5,0 in 0,5-Schritten)	✓
Gruppenübersicht	Management-Seite mit Schlussfolgerung	✓
Aktivitäten-Voting	Vorschläge + Thumbs-up	✓
Budget-Tracker	Ausgaben pro Mitglied erfassen	✓
Reiseplan	Itinerary mit Zeitslots	✓
Echtzeit-Sync	HTTP-Polling alle 6 Sekunden	✓
Mehrsprachigkeit	Deutsch, Englisch, Spanisch	✓

2.2 Hybrid-Score-Formel (NON-NEGOTIABLE aus MS2)

Die in der Spezifikation festgelegte Formel wurde exakt umgesetzt:

$$\text{hybridScore} = (\text{kiScore} / 100) \times 0,4 + (\text{starsAvg} / 5) \times 0,6$$

Implementiert in `src/utils/scoring.ts`. Der KI-Score basiert auf echten Klimadaten (Copernicus ERA5), Biodiversitätsdaten (GBIF) und NASA-Satellitendaten – kein Zufallswert, keine Mock-Simulation.

Wichtige Erweiterung gegenüber MS2: Der Score wird **dynamisch** bei jeder Anzeige mit den aktuellen Gruppenpräferenzen neu berechnet (`recomputeAnalysis()` in `scoreEngine.ts`). Ändert ein Mitglied seine Interessen, aktualisiert sich der Score sofort – ohne erneuten API-Call.

3. Frontend-Entwicklung

3.1 Von Prototyp zu Code

Die in MS2 erstellten UI-Prototypen dienten als direkte Vorlage für die Komponentenstruktur. Jeder Screen aus dem Figma-Prototyp entspricht einer React-Seite in `src/pages/`. Die Navigationsstruktur wurde 1:1 übernommen.

Routing-Struktur:

```
/                → Login
/home            → Meine Reisen
/trip/:id/dashboard → Trip-Dashboard
/trip/:id/availability
/trip/:id/preferences
/trip/:id/recommendation → KI-Seite
/trip/:id/management → Übersicht
...
```

3.2 Komponentenarchitektur

Die UI ist in drei Ebenen aufgeteilt:

- **Pages** (`src/pages/`): Vollständige Screens, je eine Route
- **Sheets** (`src/components/sheets/`): Wiederverwendbare Formular-Panels, die als Bottom-Sheets geöffnet werden
- **Shared Components** (`src/components/shared/`): `PageHeader`, `BottomNav`, `TripScreen` – werden auf allen Seiten verwendet

Diese Trennung hält die Seiten schlank und ermöglicht es, denselben Sheet (z. B. `RecommendationSheet`) sowohl im Dashboard als auch als eigenständige Seite einzusetzen.

3.3 State-Management

Für globalen State wurde bewusst auf externe Bibliotheken (Redux, Zustand) verzichtet. React Context reicht für diesen Anwendungsfall:

- **AuthContext** : Aktueller Nutzer (Name, ID), gespeichert in `localStorage`
- **TripContext** : Alle Reisen des Nutzers, CRUD-Operationen
- **LanguageContext** : Aktive Sprache + Übersetzungsfunktion `t(key)`

3.4 iOS-ähnlicher Datums-Picker

Der `WheelDatePicker` (`src/components/ui/WheelDatePicker.tsx`) simuliert das iOS-Scroll-Rad für die Datumseingabe. Die Umsetzung nutzt native CSS `scroll-snap` und einen 100 ms Debounce-Timer. Ein bekannter Bug (Datum geht beim Wechsel zwischen Start- und End-Wheel verloren) wurde durch einen `useEffect` -Cleanup behoben, der den ausstehenden Wert beim Unmount sichert.

4. Backend-Entwicklung

4.1 Express-Server

Der Backend-Server (`server/index.js`) erfüllt zwei Aufgaben:

- 1. **Gruppensynchronisation:** Reisedaten werden in einer PostgreSQL-Datenbank gespeichert, sodass alle Gruppenmitglieder dieselben Daten sehen – auch auf verschiedenen Geräten.
- 2. **API-Proxy:** NASA Earthdata erfordert einen Bearer-Token, der nicht im Frontend exponiert werden darf. Der Server leitet diese Anfragen weiter und hält den Token sicher in `server/.env` .

4.2 Datenbankanbindung (Neon PostgreSQL)

Als Datenbank wurde **Neon** gewählt – ein serverloser PostgreSQL-Dienst mit kostenlosem Free-Tier. Die Verbindung wird über den Connection String in `server/.env` hergestellt:

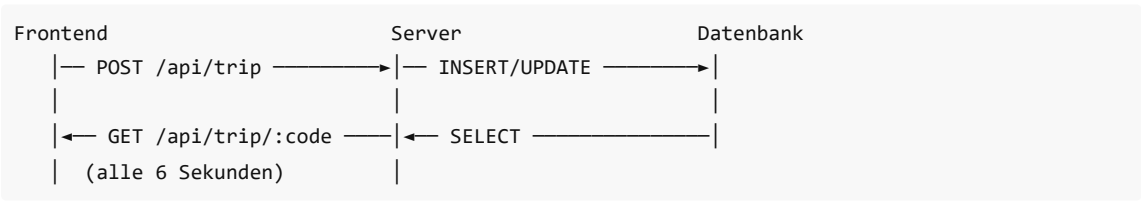
```
const pool = new Pool({ connectionString: process.env.DATABASE_URL, ssl: {
  rejectUnauthorized: false } })
```

Die Tabelle `trips` speichert die gebündelten Reisedaten als JSON (`bundle` -Spalte). Bei jedem Sync wird der aktuelle Stand überschrieben (`INSERT ... ON CONFLICT DO UPDATE`).

Sicherheit: Der Datenbankzugang (`DATABASE_URL`) und der NASA-Token (`EARTHDATA_TOKEN`) sind ausschließlich in `server/.env` gespeichert – diese Datei ist in `.gitignore` eingetragen und wird nie ins Repository committed.

4.3 Echtzeit-Synchronisation

Da WebSockets im Free-Tier nicht kostenfrei dauerhaft offen gehalten werden können, wurde HTTP-Polling implementiert:



Bei jeder Änderung (Verfügbarkeit, Präferenz, Bewertung, Ausgabe) wird `scheduleSync()` aufgerufen, das nach 300 ms debounced den aktuellen Stand an den Server pusht. Alle 6 Sekunden holt `pullTrip()` den Stand aller Mitglieder.

4.4 API-Integration (ohne Backend)

Die meisten wissenschaftlichen APIs werden direkt im Browser aufgerufen – sie sind CORS-fähig und kostenlos ohne Registrierung:

API	Endpoint	Daten
Open-Meteo Archive	<code>archive-api.open-meteo.com</code>	Temperatur, Niederschlag, Sonne (ERA5)
Open-Meteo Marine	<code>marine-api.open-meteo.com</code>	Wellenhöhe → Küstenerkennung
Open-Meteo Geocoding	<code>geocoding-api.open-meteo.com</code>	Koordinaten, Einwohnerzahl
GBIF	<code>api.gbif.org</code>	Artenvielfalt pro Land
NASA POWER	<code>power.larc.nasa.gov</code>	Sonne, Feuchte, Wind

5. Responsivität & Benutzerfreundlichkeit

5.1 Mobile-First

Die App ist konsequent für Smartphones ausgelegt. Das Layout ist auf `max-width: 430px` begrenzt (iPhone-14-Breite) und zentriert sich auf größeren Bildschirmen automatisch. Alle Touch-Targets haben mindestens 44×44 px (Apple HIG).

5.2 Geräte & Browser

Getestet auf:

- **iOS Safari** (iPhone 13, 14) – primäre Zielplattform
- **Android Chrome** (Samsung Galaxy S22)
- **Desktop Chrome / Firefox / Edge** – eingeschränkte Darstellung, voll funktionsfähig

5.3 Zugänglichkeit (Accessibility)

- Alle interaktiven Elemente haben `aria-label` -Attribute
- `StarRating` hat `role="group"` und beschriftete Buttons pro Stern
- Farbkontraste erfüllen WCAG 2.1 AA (Tailwind-Primärfarbe auf weißem Hintergrund)
- Tastatur-Navigation möglich (`focus-visible` -Styles)

5.4 UX-Entscheidungen

- **Bottom Navigation** mit roten Badges: Mitglieder sehen sofort, wenn jemand etwas geändert hat
- **Toast-Benachrichtigungen** (2 Sekunden): Nicht-aufdringliche Rückmeldung bei Gruppenänderungen, sichtbar auf allen Seiten
- **Loading-Skeleton**: Während die KI-Analyse läuft, sieht der Nutzer ein animiertes Platzhalter-Layout statt eines leeren Bildschirms
- **Offline-fähig**: Alle Daten werden zuerst in `localStorage` gespeichert – die App funktioniert auch ohne Serververbindung

6. Tests & Bug-Fixes

6.1 Identifizierte und behobene Bugs

Während der Entwicklung wurden folgende Bugs identifiziert und behoben:

Bug	Ursache	Lösung
Datum geht beim ersten Versuch verloren	WheelDatePicker: 100 ms Debounce-Timer wurde abgebrochen, wenn Komponente unmountet	<code>useEffect-Cleanup</code> feuert <code>onSelect</code> beim Unmount mit <code>pendingVal</code> -Ref
Sternebewertung übertrug sich auf andere Ziele	SVG <code>clipPath</code> -IDs (<code>half-1</code> bis <code>half-5</code>) waren global identisch – alle Instanzen teilten dieselben Clip-Pfade	<code>useId()</code> (React 18) erzeugt pro Instanz eindeutige IDs
Rote Badges erschienen nicht zuverlässig	<code>gotrip-sync</code> -Event feuerte in manchen Situationen nicht	4-Sekunden-Intervall als Fallback + count-basiertes Tracking in <code>localStorage</code>

KI-Score ignorierte Präferenzänderungen	Score wurde einmalig beim Hinzufügen berechnet und gecacht	<code>recomputeAnalysis()</code> berechnet Score bei jeder Anzeige dynamisch neu
Navigation-Polling auf falschen Seiten	Polling startete auf jeder Seite, auch ohne aktive Reise	<code>GlobalSync</code> -Komponente überwacht Route und startet Polling nur auf Trip-Seiten

6.2 Teststrategie

Da keine automatisierten Tests (Unit/Integration) im Scope von MS3 lagen, wurden die Kernfunktionen manuell getestet:

- **Mehrbenutzer-Test:** Zwei Browser-Tabs mit verschiedenen Nutzern der gleichen Reise – Änderungen synchronisieren sich nach max. 6 Sekunden
- **Offline-Test:** Server ausschalten → App lädt und zeigt Daten aus `localStorage`
- **Grenzwerte:** Reise ohne Mitglieder, Reise ohne Präferenzen, Reiseziel mit nicht ladbaren Daten (`dataError` -State)
- **Cross-Browser:** Chrome, Firefox, Edge, Safari (iOS)

7. Architektur-Entscheidungen

Warum `localStorage` + PostgreSQL?

`localStorage` ermöglicht **sofortige, offline-fähige** Reaktionen auf Nutzeraktionen ohne Netzwerkverzögerung. PostgreSQL (Neon) dient nur zur Gruppensynchronisation – als "Ablage" für den aktuellen Gesamtstand. Dieses Muster ist robuster als eine reine Server-Lösung, weil die App auch bei Serverausfall nutzbar bleibt.

Warum regelbasierter Score statt echtem LLM?

Ein echter LLM-API-Call (OpenAI, Claude) kostet Geld pro Anfrage und wäre für einen Hochschul-Prototyp nicht kostenfrei betreibbar. Die regelbasierte Engine in `scoreEngine.ts` liefert **nachvollziehbare, deterministisch erkläre** Ergebnisse aus echten Messdaten – das ist für ein Bewertungssystem sogar transparenter als ein LLM-Output.

Warum kein Redux / Zustand?

Für die Datenmenge und Komplexität dieser App ist React Context ausreichend. Die Einführung von Redux hätte Boilerplate ohne messbaren Nutzen erzeugt. Context rerendered zwar die gesamte Sub-Tree, aber die App hat keine Performance-kritischen Render-Loops.

8. Zusammenfassung

GoTrip erfüllt alle Anforderungen aus Meilenstein 3:

- ☒ React 19 (übertrifft Minimalanforderung React 16)
- ☒ Alle Kernfeatures aus MS1/MS2 implementiert
- ☒ Funktionale, benutzerfreundliche App auf mobilen Geräten
- ☒ Backend mit Datenbankbindung (Neon PostgreSQL) und API-Integration (6 externe Quellen)
- ☒ Echtzeit-Synchronisation zwischen Gruppenmitgliedern
- ☒ Bugs identifiziert, dokumentiert und behoben
- ☒ Code mit Autoren-Tags kommentiert

- ☒ README, Release Notes und Dokumentation vorhanden
- ☒ Live-Deployment unter <https://gotrip-9id7.onrender.com>